Final Project Report
Data Structures and Algorithms

Arithmetic Parsing, Heaps, and Graph Theory

Student Name: Trần Anh Tuấn
Student ID: 24521943
Course ID: CS4343.Q11.CTTT

Contents

1 Arithmetic Expressions and Parsing Logic **3**

1.1 Introduction to Notations 3

1.2 The Isomorphism of Expression Trees 3

1.3 Algorithmic Conversion: Infix to Postfix 4

1.3.1 Algorithm Logic 4

1.3.2 Handling Associativity 4

1.4 Postfix Evaluation Algorithm 4

2 The Heap Data Structure **5**

2.1 Conceptual Definition 5

2.2 Array Implementation 5

2.3 The Heapify Operation 5

2.4 Code Implementation: Heap Sort 5

3 Graph Data Structure **7**

3.1 Mathematical Definition 7

3.2 Representations and Trade-offs 7

3.2.1 1. Adjacency Matrix 7

3.2.2 2. Adjacency List 7

3.3 Graph Traversals 7

3.3.1 Breadth-First Search (BFS) 7

3.3.2 Depth-First Search (DFS) 8

3.4 Code Implementation: BFS and DFS Traversals 8

4 Conclusion **10**

1 Arithmetic Expressions and Parsing Logic

1.1 Introduction to Notations

In computer science, the evaluation of mathematical expressions is a non-trivial task. While humans rely on visual cues and memorized rules (like BODMAS/PEMDAS) to interpret expressions, computers require a strictly deterministic sequence of operations.

1. **Infix Notation ($A + B$):** The operator sits between operands. This is human-readable but computationally expensive to parse due to ambiguity and parentheses.
2. **Prefix Notation ($+AB$):** Also known as Polish Notation. The operator precedes operands.
3. **Postfix Notation ($AB+$):** Also known as Reverse Polish Notation (RPN). The operator follows operands.

1.2 The Isomorphism of Expression Trees

The relationship between these notations is best understood through **Expression Trees**. An expression tree is a binary tree where leaves are operands and internal nodes are operators.

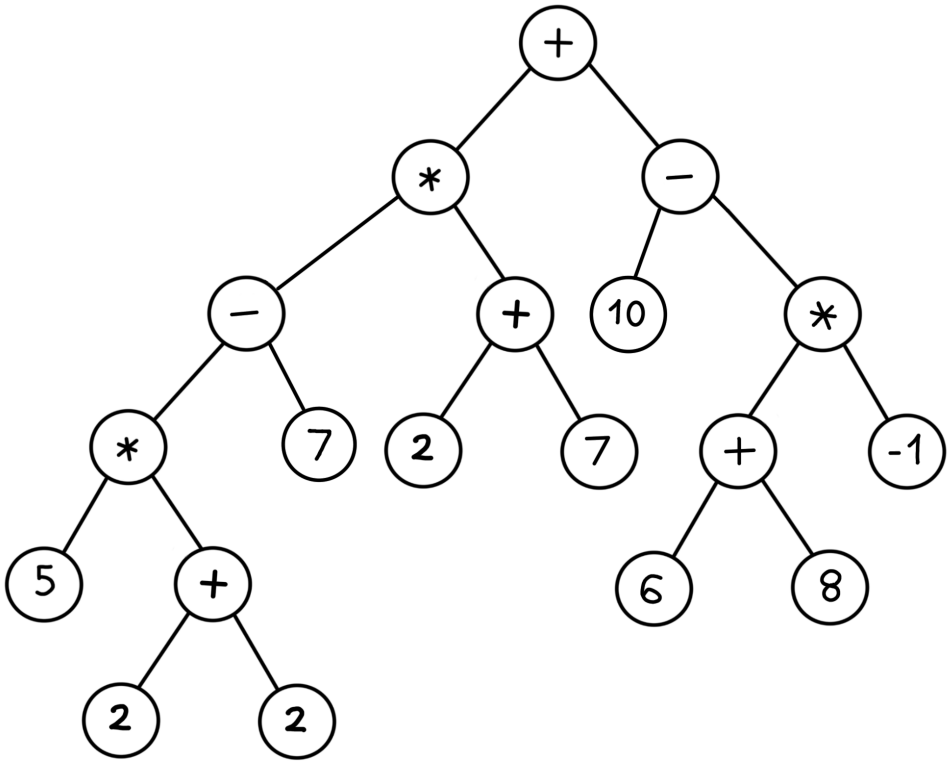

Figure 1: Binary Expression Tree Structure

Traversal methods yield the different notations:

- **Preorder Traversal (Root → Left → Right)** yields **Prefix**.
- **Inorder Traversal (Left → Root → Right)** yields **Infix**.
- **Postorder Traversal (Left → Right → Root)** yields **Postfix**.

This proves that all three notations are merely different serializations of the same underlying mathematical structure.

1.3 Algorithmic Conversion: Infix to Postfix

Computers prefer Postfix because it eliminates the need for parentheses. The conversion is typically performed using the **Shunting-yard algorithm** (Dijkstra).

1.3.1 Algorithm Logic

We utilize a Stack to temporarily store operators. The core rules are: 1. Operands are output immediately. 2. If an incoming operator has *lower or equal* precedence than the operator on top of the stack, pop the stack to the output before pushing the new operator. 3. Left Parenthesis '(' is pushed; Right Parenthesis ')' pops everything until '(' is found.

1.3.2 Handling Associativity

A critical edge case is **Right-Associativity** (e.g., Exponentiation 2^{3^4}). Unlike addition $((A + B) + C)$, exponentiation is evaluated right-to-left $(A^{(B^C)})$.

- For Left-Associative operators $(+, -, *, /)$, pop if stack precedence is $\geq$ current.
- For Right-Associative operators $(\wedge)$, pop only if stack precedence is strict $>$ current.

1.4 Postfix Evaluation Algorithm

Once converted, evaluating Postfix is efficient $O(n)$.

Algorithm 1 Postfix Evaluation

```
Create Stack S
for each token T in Expression do
    if T is Number then
        Push T to S
    else                                        ▷ T is Operator
        Val2 ← S.pop()
        Val1 ← S.pop()
        Res ← Compute(Val1, T, Val2)
        Push Res to S
    end if
end for
return S.top()
```

2 The Heap Data Structure

2.1 Conceptual Definition

A Heap is a specialized Tree-based data structure that satisfies two properties:

1. **Shape Property:** It is a *Complete Binary Tree*. All levels are filled except possibly the last, which is filled left-to-right.
2. **Heap Property:**
   - **Max-Heap:** Parent nodes are always $\geq$ children.
   - **Min-Heap:** Parent nodes are always $\leq$ children.

2.2 Array Implementation

Heaps are implemented as arrays for cache efficiency. For a node at index i:

$$\begin{aligned} \text{Parent} &= \lfloor (i-1)/2 \rfloor \\ \text{Left Child} &= 2i+1 \\ \text{Right Child} &= 2i+2 \end{aligned}$$

2.3 The Heapify Operation

The structural integrity of the heap is maintained by `Heapify`.

- **Heapify-Up (Swim):** Used on Insertion. The new element at the bottom "swims" up until the heap property is restored.
- **Heapify-Down (Sink):** Used on Deletion. The root is replaced by the last element, which then "sinks" down to its correct position.

2.4 Code Implementation: Heap Sort

Heap Sort is an efficient comparison-based sorting algorithm with $O(n \log n)$ complexity.

```cpp
#include <bits/stdc++.h>
using namespace std;

// Maintain Max-Heap property
void heapify(vector<int>& arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

```

```
    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(vector<int>& arr) {
    int n = arr.size();
    // Build heap (rearrange array)
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Extract elements one by one
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]); // Move current root to end
        heapify(arr, i, 0);   // Call max heapify on reduced heap
    }
}
```

Listing 1: C++ Implementation of Heap Sort

3 Graph Data Structure

3.1 Mathematical Definition

A Graph G is an ordered pair (V, E), where V is a set of vertices and E is a set of edges connecting the vertices. Graphs can be Directed $(u \to v)$ or Undirected $(u - v)$, and Weighted or Unweighted.

3.2 Representations and Trade-offs

There are two primary ways to store a graph in memory.

3.2.1 1. Adjacency Matrix

A 2D array $A[V][V]$ where $A[i][j] = 1$ (or weight) if an edge exists.

- **Space:** $O(V^2)$.
- **Pros:** $O(1)$ edge lookups. Good for dense graphs.
- **Cons:** Wastes memory for sparse graphs.

3.2.2 2. Adjacency List

An array of lists/vectors where index i stores all neighbors of vertex i.

- **Space:** $O(V + E)$.
- **Pros:** Memory efficient for sparse graphs (most real-world networks).
- **Cons:** Edge lookup is $O(degree(V))$.

3.3 Graph Traversals

Traversing a graph involves visiting all nodes reachable from a source.

3.3.1 Breadth-First Search (BFS)

BFS explores neighbors layer-by-layer.

- **Data Structure:** Queue (FIFO).
- **Application:** Shortest path in unweighted graphs (e.g., GPS, Social Network degrees of separation).

3.3.2 Depth-First Search (DFS)

DFS explores as deep as possible along each branch before backtracking.

- **Data Structure:** Stack (LIFO) or Recursion.
- **Application:** Cycle detection, Topological Sorting, Solving Mazes.

3.4 Code Implementation: BFS and DFS Traversals

```cpp
#include <bits/stdc++.h>
using namespace std;

class Graph {
    int V;
    vector<vector<int>> adj;

    // Recursive helper function for DFS
    void DFSUtil(int v, vector<bool>& visited) {
        // Mark the current node as visited and print it
        visited[v] = true;
        cout << v << " ";

        // Recur for all the vertices adjacent to this vertex
        for (int i : adj[v]) {
            if (!visited[i])
                DFSUtil(i, visited);
        }
    }

public:
    Graph(int V) {
        this->V = V;
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u); // Undirected
    }

    // Breadth-First Search
    void BFS(int start) {
        vector<bool> visited(V, false);
        queue<int> q;

        visited[start] = true;
        q.push(start);

        cout << "BFS Traversal: ";
        while(!q.empty()) {
```

```
            int u = q.front();
            q.pop();
            cout << u << " ";

            for(int v : adj[u]) {
                if(!visited[v]) {
                    visited[v] = true;
                    q.push(v);
                }
            }
        }
        cout << endl;
    }

    // Depth-First Search
    void DFS(int start) {
        vector<bool> visited(V, false);
        cout << "DFS Traversal: ";
        DFSUtil(start, visited);
        cout << endl;
    }
};
```

Listing 2: Graph Implementation with BFS and DFS

4 Conclusion

This technical report has provided a deep analysis of three foundational areas of Computer Science.

- **Expressions:** We concluded that while Infix notation serves human readability, Postfix notation is superior for machine execution. The isomorphism between expression trees and tree traversals provides the theoretical backing for this conversion.
- **Heaps:** We demonstrated that Heaps are the optimal structure for Priority Queues. By utilizing the array-based implementation of a Complete Binary Tree, we achieve $O(\log n)$ efficiency for insertion and deletion, which is critical for operating system scheduling.
- **Graphs:** We explored the trade-offs between Matrix and List representations. We established that BFS is the algorithm of choice for shortest-path problems in unweighted networks, while DFS is preferred for connectivity and path-existence problems.

Mastery of these data structures allows engineers to optimize memory usage and processing time, forming the backbone of efficient software development.